

Guía Práctica 5: Congestión

Tomás F. Melli

October 2025

Índice

1	Ejercicio 1	2
2	Ejercicio 2	2
3	Ejercicio 3	3
4	Ejercicio 4	3
5	Ejercicio 5	4
6	Ejercicio 6	5
7	Ejercicio 7	5
8	Ejercicio 8	5
9	Ejercicio 9	6
10	Ejercicio 10	7
11	Ejercicio 11	9
12	Ejercicio 12	9

1 Ejercicio 1

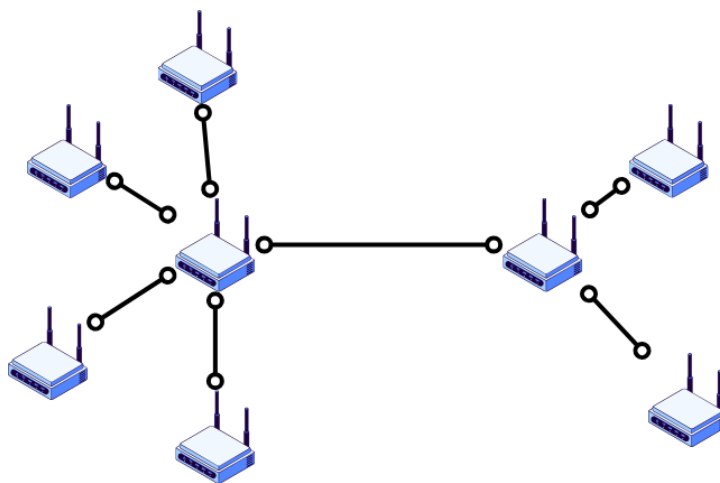
Dado un protocolo de nivel de red basado en forwarding de datagramas (i.e.: IP):

¿Se puede realizar control de congestión utilizando la técnica de prealocación de buffers?

En primer lugar, la *prealocación de buffers* es una técnica de gestión de memoria usada dentro de la pila de red (en el kernel o en un router) para reservar espacio de antemano donde se almacenarán los paquetes entrantes o salientes antes de que realmente lleguen o se necesiten. El tema es que como IP no es orientado a conexión y no tiene sesiones ni sabe si el flujo está activo, el kernel o el router no tienen chance de planificar buffers por conexión. Como consecuencia, siempre tiene que tener buffers disponibles. Ahora bien, en un escenario de muchísima recepción de paquetes con bajo throughput de procesamiento de salida de esos paquetes conlleva a una pérdida de los paquetes siguientes porque no hay lugar para alojarlos. Conclusión, en escenarios de bajo flujo está bien porque llegan y tienen memoria disponible, pero en cuanto se pone picante la cosa, flaquea.

¿Los protocolos de ruteo pueden por sí solos evitar la congestión en una red?

Recordemos de los temas del primer parcial que los protocolos de ruteo son los mecanismos que tienen los routers para elegir la mejor ruta, vimos RIP y OSPF. Supongamos la siguiente topología :



Los protocolos de ruteo utilizan como criterio de selección de ruta, la distancia, la velocidad de transmisión, pero no están diseñados para gestionar la congestión. Pueden sí, recalcular rutas en caso de fallos, hasta incluso en casos de congestión si existen rutas alternativas desviar el tráfico, pero es una medida muy limitada, en particular en topologías como la del ejemplo.

En el caso de disponer de buffers con memoria infinita en los routers. ¿Se puede evitar la congestión en una red?

Esta es sólo una variable que se soluciona. La congestión se puede pensar como una saturación de la capacidad para transmitir todos los paquetes enviados. Con esto en mente, el ancho de banda es otro de los factores, por tanto, si miramos la topología de ejemplo, por más memoria que tengamos, incluso con una capacidad de procesamiento locura, tenemos una limitación física del enlace que no nos va a permitir transmitirlos y caeremos en un escenario de congestión de red.

2 Ejercicio 2

El control de congestión de TCP usa la CWND como una estimación de la congestión presente en la red.

¿En qué casos no es necesario un control de congestión?

En redes privadas, controladas y de baja carga, en redes con ancho de banda fijo y tráfico controlado.

Mencione dos situaciones en las que la CWND no sea representativa de la congestión de la red. ¿Qué medidas deberían tomarse ante estas situaciones?

1. Errores en el canal: TCP piensa que cuando se pierde un paquete es porque hay congestión, pero puede ser por alguna cuestión física.

2. Demoras en el procesamiento de paquetes en un período corto: el receptor le puede estar costando procesar los paquetes y darles salida y por tanto, a través de RWND modifica el valor de CNWD. Este valor se proyecta por un rato hasta actualizarse, sin embargo, no refleja una congestión real.

3 Ejercicio 3

CWND y Ssthresh son las dos variables que deben regularse en el control de congestión de TCP.

¿Qué valores deberían tener cuando la conexión recién comienza?

El valor de Ssthresh nos define el umbral que determina cuándo TCP debe pasar de la fase de Slow Start a Congestion Avoidance. Por ello, este valor puede ser arbitrariamente grande al principio. Por el contrario, el valor de CWND podemos inicializarla como $2 \times SMSS$ que es el valor de *IW* (initial window). La idea es que este valor no sea demasiado alto para no saturar de una la red. Luego, la llegada de ACKs actuarán como retroalimentación positiva, permitiendo que CWND crezca exponencialmente hasta alcanzar Ssthresh.

¿Cómo se modifican ante un time out?

En caso de ocurrir un time-out, la ventana de congestión se reinicia con:

$$CWND = LW(1 \ SMSS)$$

El umbral de Slow Start se ajusta a:

$$Ssthresh = \max(FlightSize/2, 2SMSS)$$

Se comienza de nuevo la fase de Slow Start. El time-out actúa como una retroalimentación negativa, indicando que la red está saturada y que se debe reducir temporalmente la tasa de envío para evitar mayor congestión.

Slow Start se usa cuando no se conoce el estado de la congestión de la red. ¿Por qué no esta bien empezar con una ventana arbitrariamente grande?

El tema es que si enviamos una gran cantidad de paquetes como la advertised window, podemos saturar el router ya que depende de la capacidad de los buffers.

¿Cuánto debe incrementarse la ventana de congestión por RTT durante Congestion Avoidance?

Una vez que CWND alcanza el Ssthresh, el crecimiento deja de ser exponencial y pasa a lineal. La CWND aumenta aproximadamente un SMSS por RTT:

$$CWND+ = \frac{SMSS^2}{CWND}$$

Esta estrategia permite un aumento controlado de la ventana y reduce el riesgo de congestión.

4 Ejercicio 4

En una conexión TCP recién establecida se envían estos segmentos en el siguiente orden y con los siguientes tamaños:

100 bytes ; 150 bytes ; 200 bytes ; 300 bytes

Todos los segmentos llegan al receptor sin errores pero el segundo segmento llega último.

Indicar hasta qué byte reconoce el receptor en cada ACK.

TCP solo reconoce (ACK) los bytes contiguos recibidos correctamente. Si llega algo fuera de orden, TCP lo almacena (buffer) pero no aumenta el ACK hasta que recibe todos los bytes faltantes anteriores.

Momento	Segmento que llega	Último byte contiguo recibido	ACK enviado
1	#1 (1-100)	100	ACK = 101
2	#3 (251-450)	100	ACK = 101
3	#4 (451-750)	100	ACK = 101
4	#2 (101-250)	750	ACK = 751

Indicar cuál es el valor del **SSTHRESH**, posterior a la recepción del ACK que indica que los cuatro segmentos llegaron a destino.

No hay motivo para modificar el valor de **SSTHRESH** ya que no hubo timeout, ni pérdida de paquetes, ni congestión en la red.

Indicar cuál es el valor del **CWND**, posterior a la recepción del ACK que indica que los cuatro segmentos llegaron a destino.

Recordemos la fórmula en el escenario de Slow Start:

$$CWND+ = \min(N, SMSS) \text{ por cada ACK recibido}$$

donde N es la cantidad de bytes reconocidos por el ACK.

Evento	Descripción	Tipo de ACK	CWND (en MSS)
Inicio	Conexión recién establecida	–	1
1	Llega ACK del primer segmento (ACK = 101)	Nuevo ACK	2
2	Llegan dos ACK duplicados (ACK = 101 repetido)	Duplicados	2
3	Llega ACK final (ACK = 751) que confirma todos los segmentos	Nuevo ACK	3

5 Ejercicio 5

Suponga que, para una conexión, cuatro ACKs fueron devueltos con los siguientes RTTs: [80ms, 20ms, 100ms, 90ms]. Recordamos:

- Fórmula de estimación:

$$RTT[i+1] = \alpha \cdot RTT[i] + (1 - \alpha) \cdot SRTT_i$$

- RTO simple:

$$RTO[i] = 2 \cdot RTT[i]$$

- Primer RTT medido: Tomar el RTT inicial como la primer medición

$$RTT[0] = SRTT[0] = 80 \text{ ms}$$

Calcule el RTO luego de la llegada de cada ACK usando $\alpha = 1,0$.

$$RTT[i+1] = 1 \cdot RTT[i] + 0 \cdot SRTT_i = RTT[i]$$

Entonces:

$$RTT = [80, 80, 80, 80] \text{ ms}$$

$$RTO = 2 \times RTT = [160, 160, 160, 160] \text{ ms}$$

No hay timeout, ya que todos los ACKs llegan antes del RTO correspondiente.

Repita el cálculo para $\alpha \in [0,8, 0,6, 0,4, 0,2]$.

Análogo

¿Se produce algún time-out en alguno de los escenarios anteriores?

Hay que mirar si todos los ACKs llegan antes del RTO correspondiente.

6 Ejercicio 6

Considere el efecto de usar Slow Start en una conexión TCP recién establecida ($IW = 2 * SMSS$, $SSTHRESH = 64KB$, $SMSS = 2KB$), que tiene un RTT de 10 mseg y sin congestión ni errores presentes en la red. Si la RWND es de 24 KB,

¿Cuánto tiempo transcurre antes de que pueda ser enviada la primera ventana de recepción llena?

La idea aca es simular el intercambio de mensajes teniendo en cuenta que el Ttx de un ráfaga es mucho más chico que el Tprop de la conexión ($Ttx \ll Tprop$). Esta es una condición fuerte pero asegura que los envíos se den de a ráfagas, lo cual simplifica la resolución del ejercicio, de manera de poder capturar la dinámica de intercambio de paquetes.

En el estado inicial, la conexión arranca con $CWND = IW = 2 * SMSS$. En el primer RTT se envía una ráfaga de dos segmentos llegando a un total de 4KB de datos en vuelo. Al momento de recibir cada ACK de vuelta, la ventana de congestión se incrementa como $CWND += \min(N, SMSS)$ (Cabe aclarar que en este caso $N=SMSS$). Esto deja la $CWND = 4 * SMSS = 8KB$.

Así, la próxima ráfaga será de 8KB y volverán 4 ACKs de 2KB cada uno, por lo que la ventana llegará a 16KB al comienzo del próximo envío, que sería el tercero. El próximo envío (el tercero), se envían 16KB y vuelven 8 ACKs de 2KB cada uno $\Rightarrow CWND=32KB$. A partir de acá, en el 4to RTT, el envío continúa con $FlightSize = \min(RWND, CWND) = \min(24, 32) = 24$, momento en el cual se dice que se envía la primera ventana de recepción llena.

Esto puede quedar más claro usando una tabla como la siguiente, donde en cada línea está el estado de las variables de la conexión que necesitamos para resolver el ejercicio:

RTT	CWND	RWND	FlightSize
1	4KB	24KB	4KB
2	8KB	24KB	8KB
3	16KB	24KB	16KB
4	32KB	24KB	24KB

7 Ejercicio 7

Según el [RFC 5681](#),

Durante slow start, la ventana de congestion DEBERÍA incrementarse usando $CWND+ = \min(N, SMSS)$. Sin embargo, según el RFC 2581, se usaba la fórmula $CWND+ = SMSS$. ¿A que se debió este cambio?

CWND se incrementa de acuerdo a la cantidad de datos realmente reconocidos, en lugar de un incremento rígido de SMSS por cada ACK recibido, lo que puede hacer que el crecimiento de la ventana de congestión sea más controlado. Este cambio ayuda a mantener el incremento de CWND más eficiente en redes donde el receptor puede recibir varios segmentos sin enviar un ACK por cada uno. Así, CWND se incrementa de acuerdo a la cantidad de datos realmente reconocidos, en lugar de un incremento rígido de SMSS por cada ACK recibido, lo que puede hacer que el crecimiento de la ventana de congestión sea más controlado. N es la cantidad de bytes que no habían sido reconocidos anteriormente pero se están reconociendo en el ACK recibido. El cambio surge para que el algoritmo sea más robusto contra receptores que se portan mal e intentan inducir al emisor a inflar artificialmente su CWND, andando múltiples ACKs por un único segmento de TCP.

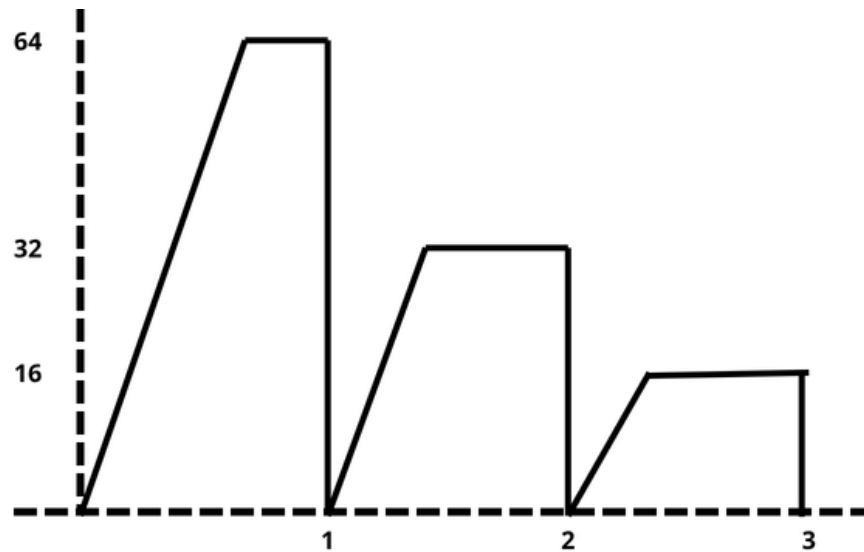
En una conexión TCP, el receptor PUEDE demorar los ACKs hasta haber recibido 2 SMSS del emisor o hasta 500ms, lo que suceda primero. ¿Qué impacto tienen estas demoras en el emisor?

Pueden haber retransmisiones innecesarias, y un aumento del RTT, que hace que se estime uno más largo. Además el CWND no aumenta de manera óptima, ya que depende de los ACKs. Esto significa que el emisor podría no incrementar su tasa de envío tan rápido como lo haría sin las demoras de ACKs. Esto puede afectar la rapidez con la que el emisor detecta cambios en el estado de congestión de la red. Además, las demoras en los ACKs pueden reducir la eficiencia del canal de transmisión, ya que el emisor podría quedar esperando ACKs antes de poder enviar más datos

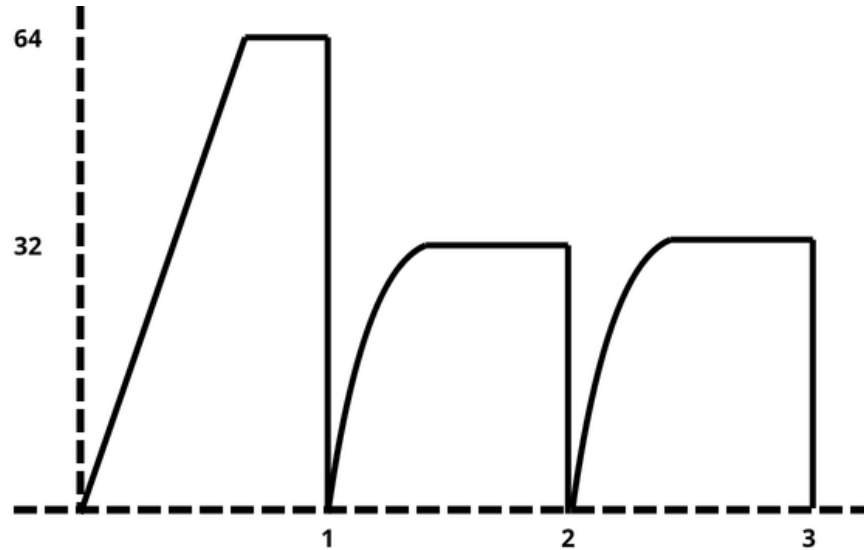
8 Ejercicio 8

Grafique CWND en función del tiempo para una conexión TCP que pierde paquetes cada 1 seg para los siguientes casos detallando los valores de SSTHRESH:

Sin Fast Retransmit / Fast Recovery



Con Fast Retransmit / Fast Recovery



9 Ejercicio 9

Una conexión TCP tiene que enviar 1GB de datos. Suponiendo que no se producen errores en la transmisión de los datos y que las únicas pérdidas de paquetes son por congestión. ¿Calcular cuánto tiempo tarda la conexión en enviar todos los datos para los siguientes casos?

Asumir: si una ráfaga causa congestión se pierden todos los segmentos de la misma antes de llegar al receptor y que la RWND vale siempre 64KB

Congestión con 32KB por RTT

1. 1º RTT: Envía 4 KB (2 SMSS $\rightarrow$ llegan 2 ACK). CWND += 2*2 KB (se suman 2 veces SMSS, $N = \min(N, SMSS)$).
2. 2º RTT: Envía 8 KB y CWND += 8 KB.
3. 3º RTT: Envía 16 KB y CWND += 16 KB.
4. 4º RTT: No supera 64 KB pero se produce congestión $\rightarrow$ time-out.
 - SSTHRESH = $\max(\text{FlightSize}/2, 2 * SMSS) = \max(16 \text{ KB}, 4 \text{ KB}) = 16 \text{ KB}$
 - CWND = 2 KB

5. 5º RTT: Envía 2 KB, $CWND += 2 \text{ KB}$.
6. 6º RTT: Envía 4 KB, $CWND += 4 \text{ KB}$.
7. 7º RTT: Envía 8 KB, $CWND += 8 \text{ KB} \rightarrow CWND = 16 \text{ KB}$.
8. 8º RTT: $CWND \geq Ssthresh \rightarrow$ pasa a *Congestion Avoidance*. Envía 16 KB, $CWND += \lceil (SMSS^2 / CWND) * 8 \rceil = 18 \text{ KB}$
9. 9º RTT: Envía 18 KB, $CWND += \lceil (SMSS^2 / CWND) * 9 \rceil = 20 \text{ KB}$ (para simplificar: suponemos que aumenta 1 SMSS = 2 KB por RTT)
10. 10º RTT: Envía 20 KB, $CWND += \lceil (SMSS^2 / CWND) * 10 \rceil = 22 \text{ KB}$
11. ... y así sucesivamente hasta alcanzar 32 KB, donde ocurre otro time-out y se vuelve a *slow start*.

Congestión con 16KB por RTT

Análogo

Sin congestion (i.e.: una conexión establecida entre hosts de la misma LAN)

Arranca y sigue con slow start hasta que envía $2 + 4 + 8 + 16 + 32 + 64 = 126 \text{ KB}$. Ahí hace 6 RTT. Luego, como $CWND \geq Ssthresh$, pasa a congestion avoidance. Como nunca ocurre un timeout, sigue enviando así hasta terminar 1GB.

10 Ejercicio 10

En una conexión recién establecida con $RTT = 200 \text{ ms}$, el host receptor siempre anuncia una Advertised Window de 16 KB. La red está cargada al punto que si una ráfaga fuera de 16 KB o más, se perderían todos los segmentos de la misma.

1. ¿Cuánto vale la $CWND$ luego de enviar un archivo de 40 KB?
2. 3 segundos después del envío del archivo, se envía otro archivo de 30 KB. ¿Cuánto tiempo tarda?

Solución

Como se mencionó previamente, una forma sencilla de encarar estos problemas es definir una tabla donde se lleva la cuenta de todas las variables que nos interesan para poder resolver el problema. Para esto, debemos decidir qué variables involucrar. Primero, el RTT es el elemento principal que lidera la evolución de la conexión. Segundo, las ventanas del control de flujo (RWND) y del control de congestión (CWND), cuyo mínimo define la máxima ventana disponible para enviar. En tercer lugar, también debemos llevar el Ssthresh, dado que, ante la pérdida de una ráfaga, esta variable se actualizará para marcar el fin de Slow Start; esta variable comienza valiendo un valor arbitrariamente alto, por ejemplo 64 KB. Por último, llevamos cuenta de los datos en vuelo (FlightSize), dado que el control de congestión lo tiene en cuenta para ajustar la ventana, y de los datos enviados hasta el momento (LastByteSent o LBS), que marca cuánto hemos enviado con éxito hasta el momento.

RTT	CWND	RWND	SSTHRESH	FlightSize	LBS	Comentarios
1	4KB	16KB	64KB	4KB	4KB	La CWND inicia en $IW = 2 * SMSS$ y tomamos $SMSS = 2KB$. Siempre, $RWND = 16KB$ porque, en este caso, la Advertised Window nunca cambia.
2	8KB	16KB	64KB	8KB	12KB	Al principio aumenta con Slow Start, dado que el SSTHRESH es arbitrariamente alto.
3	16KB	16KB	64KB	16KB	28KB	Aca los datos están en vuelo, y no hay respuesta de ACKs dado que se pierden todos los segmentos de la ráfaga.
4	16KB	16KB	64KB	16KB	28KB	TIME OUT, dado que $RTO = 2 * RTT$.
5	2KB	16KB	8KB	2KB	14KB	La ventana se actualiza a $LW = 1 * SMSS = 2KB$ y el $SSTHRESH = \text{Max}(\text{FlightSize}/2, 2 * SMSS)$.
6	4KB	16KB	8KB	4KB	18KB	Aca sigue con Slow Start hasta que la CWND alcance el SSTHRESH.
7	8KB	16KB	8KB	8KB	26KB	A partir de aca se aumenta con Congestion Avoidance a razón de un SMSS por RTT.
8	10KB	16KB	8KB	10KB	36KB	Aca se aproxima el aumento de la CWND. Tomamos 1 SMSS por RTT.
9	12KB	16KB	8KB	4KB	40KB	Solo faltan 4KB para llegar a los 40KB.

Se puede decir que la CWND termina en 12KB luego de enviar todos los datos pero, la transmisión con éxito, termina luego de la llegada de los ACKs que faltan. Por aproximación tomamos que este último RTT, la CWND, aumentó otro SMSS y termina valiendo **14KB**.

Para el segundo inciso, la idea es acordarse que, luego de un RTO sin transmitir, la CWND, se reinicia a $RW = \min(IW, CWND) = 4KB$. Una vez tenido esto en cuenta, se puede ver cuanto tardaría en enviarse los próximos 30KB, usando la tabla:

RTT	CWND	RWND	SSTHRESH	FlightSize	LBS	Comentarios
1	4KB	16KB	8KB	4KB	4KB	Comienzo a contar en terminos de RTT desde 1 nuevamente. SSTHRESH se mantiene como venía hasta ahora porque no se aclara que deba cambiar.
2	8KB	16KB	8KB	8KB	12KB	Como $CWND < SSTHRESH$ aumento con Slow Start.
3	10KB	16KB	8KB	10KB	22KB	Aca comienzo Congestion Avoidance.
4	12KB	16KB	8KB	8KB	30KB	Al final de este RTT termina con éxito la transmisión.

Hicieron falta 4 RTTs para enviar los 30KB. La respuesta sería: $4 * 200ms = 0,8seg$

11 Ejercicio 11

Dada una conexión TCP recién establecida entre dos host para la cual el RTT es de 50ms. Los dos host están separados por un sólo router que también conecta otras redes y está cargado a tal punto que cada vez que una ráfaga de paquetes es de 20KB o más, se descartan todos los paquetes de la ráfaga. El host emisor tiene que enviar un archivo bastante grande que se está transmitiendo por horas y el host receptor siempre anuncia una *AdvertisedWindow* de 28KB.

a. Si se define que una conexión alcanza el *estado estacionario* en el momento que el Ssthresh converge a un valor a partir del cual ya no cambia más. ¿Cuánto tiempo tarda la conexión en alcanzar el estado estacionario? ¿Cuál es el valor del Ssthresh en dicho momento?

b. Finalizada la transferencia, se cierra la conexión, y se inicia una nueva en la que el host receptor siempre anuncia una *AdvertisedWindow* de 18KB. Si esta nueva conexión tuviera que transferir el mismo archivo, ¿tardaría más o menos tiempo que la anterior? (*Suponer las mismas condiciones de congestión en el router*)

Solución

RTT	CWND	RWND	Ssthresh	FlightSize	LBS	Comentarios
1	4	28	64	4	4	-
2	8	28	64	8	12	-
3	16	28	64	16	28	-
4	32	28	64	28	56	-
5	32	28	64	28	56	TIME OUT
6	2	28	14	2	30	-
7	4	28	14	4	34	-
8	8	28	14	8	42	-
9	16	28	14	16	58	-
10	18	28	14	18	76	-
11	20	28	14	20	96	-
12	20	28	14	20	96	TIME OUT
13	2	28	10	2	78	ACA EL Ssthresh ADOPTA EL ULTIMO VALOR Y NO CAMBIA MAS
14	4	28	10	4	82	-
15	8	28	10	8	90	-
16	12	28	10	12	102	-
17	14	28	10	14	116	-
18	16	28	10	16	132	-
19	18	28	10	18	150	-
20	20	28	10	20	170	-
21	20	28	10	20	170	TIME OUT
22	2	28	10	2	152	ACA EL Ssthresh YA DA LO MISMO

Bajo este diagrama la respuesta es, a partir del RTT 13 $\rightarrow 13 * 50ms = 650ms$

Para el segundo ítem, la MaxWindow del host emisor se mantendrá constante en 18KB regida por la RWND. El host receptor mantiene al emisor enviando datos justo por debajo del umbral de congestión. Esto hace que no se pierdan paquetes por congestión y que en cada RTT se envíen 18KB. En promedio, se enviarán más datos por RTT que en la conexión anterior, donde la red si llega a congestionarse y TCP ejecuta los mecanismos de recuperación que enlentecen el envío de datos.

12 Ejercicio 12

todo